Erweitern und Festigen der erlernten Konzepte

Oktober 2025

Plan für die Woche

Montag

- Kleinere Themen abarbeiten

Dienstag

- Overload vs Override
- Besprechung PDF

Mittwoch

- Lernstandskontrolle
- Abstrakte Klassen

Donnerstag

- Einführung in Interfaces
- Wie fandet ihr die Woche?
- Update Selbsteinschätzung 811.

Freitag

- Tag der Deutschen Einheit

Plan für heute

- Konzepte der Objektorientierten Programmierung
- Wiederholung Kapselung
- Einführung in die Abstraktion
  - Abstrakte Klassen

Konzepte der OOP

- OOP steht für objektorientierte Programmierung

- Konzepte:
  - Kapselung (encapsulation)
  - Abstraktion (abstraction)
  - Vererbung (inheritance)
  - Polymorphismus (polymorphism)

Wiederholung Kapselung

Kapselung:

Zugriff auf private Felder über Getter/Setter

→ schützt Daten, erlaubt Validierung

Konzepte der OOP

- OOP steht für objektorientierte Programmierung

- Konzepte:
  - ~~Kapselung (encapsulation)~~
  - **Abstraktion (abstraction)**
  - Vererbung (inheritance)
  - Polymorphismus (polymorphism)

Abstrakte Klassen

FESTIGEN UND ERWEITERN DES WISSENS

Was sind abstrakte Klassen?

- sind „normale“ Klassen, die jedoch nicht instanziiert werden können
  - Dienen also nicht zur Bildung von Objekten
  - Haben dennoch einen Konstruktor!
- dienen als Basis
  - Definieren, welche allgemeinen Eigenschaften und Fähigkeiten ein Typ haben soll
  - Mit den Subklassen(Vererbung) kann diese Klasse um Eigenschaften erweitert werden

Reminder: Vererbung wird immer mit dem Schlüsselwort **extends** angegeben

Inhalt einer abstrakten Klasse

- können Variablen halten
- können einen Konstruktor oder mehrere Konstruktoren haben
- eine abstrakte Klasse kann sowohl **abstrakte** als auch **konkrete** Methoden beinhalten

Abstract - Keyword

```
abstract class Tier {
}
```

Wenn kein Access-Modifier vor einem Datenfeld oder Methode steht, dann gilt package-private!
(für Vererbung wichtig)

Inhalt einer abstrakten Klasse

```
abstract class Tier {

    void makeSound(){
        System.out.println("Tier macht default-sound");
    }

}
```

Inhalt einer abstrakten Klasse

– konkrete Methoden:

Abstrakte Klasse

Konkrete Methode mit Methodenkörper

```
abstract class Tier {

    void makeSound(){
        System.out.println("Tier macht default-sound");
    }

}
```

Inhalt einer abstrakten Klasse

```
abstract class Tier {
    abstract void makeSound();
}
```

Inhalt einer abstrakten Klasse

- abstrakte Methode:
  - Methoden können NICHT private sein
  - Sie sind dafür da, dass sie in der Subklasse überschrieben werden (@Override)
    - Wenn sie private wären, dann könnte man sie nicht vererben

Abstract - Keyword

```
abstract class Tier {
  abstract void makeSound();
}
```

Semikolon direkt nach Methodensignatur

Keinen Methodenkörper!

Inhalt einer abstrakten Klasse

Abstrakte Klasse

Sowohl abstrakte als auch konkrete Methoden akzeptabel!

```
abstract class Tier {
    abstract void makeSound();
    void move(){
        System.out.println("Es rennt");
    }
}
```

Was sind abstrakte Klassen? - Zusammenfassung

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, **muss** die Klasse selbst als abstrakt markiert werden!
  - nur public oder protected für abstrakte Methoden

Warum einen Konstruktor in abstrakten Klassen?

- zur Initialisierung von Variablen

```
abstract class Fahrzeug {
    String hersteller;

    public Fahrzeug(String hersteller) {
        this.hersteller = hersteller;
    }

    abstract void fahren();
}

class Auto extends Fahrzeug {
    public Auto(String hersteller) {
        super(hersteller);
    }

    void fahren() {
        System.out.println(hersteller + " Auto fährt.");
    }
}
```

Ist der Code valide?

```
abstract class Tier {
    void schlafen() {
        System.out.println("Das Tier schläft.");
    }
}
```

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

Ist der Code valide?

```
abstract class Tier {
    void schlafen() {
        System.out.println("Das Tier schläft.");
    }
}
```

Ja!

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

Ist der Code valide?

```
class Fahrzeug {
    abstract void fahren();
}
```

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

Ist der Code valide?

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

```
class Fahrzeug {
    abstract void fahren();
}
```

Nein! Sobald eine abstrakte Methode in einer Klasse auftritt, muss die Klasse selbst als abstrakt markiert werden!

Ist der Code valide?

```
abstract class Lebewesen {
    void atmen() {
        System.out.println("Atmet Luft.");
    }
}
```

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

Ist der Code valide?

```
abstract class Lebewesen {
    void atmen() {
        System.out.println("Atmet Luft.");
    }
}
```

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

Passt! Abstrakte Klassen können auch konkrete Methoden haben

Ist der Code valide?

```
abstract class Pflanze {
    abstract void wachsen() {
        System.out.println("Die Pflanze wächst.");
    }
}
```

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

Ist der Code valide?

```
abstract class Pflanze {
    abstract void wachsen() {
        System.out.println("Die Pflanze wächst.");
    }
}
```

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

Abstrakte Methoden dürfen keinen Methodenkörper haben!

Ist der Code valide?

```
abstract class Werkzeug { }
public class Main {
    public static void main(String[] args) {
        Werkzeug w = new Werkzeug();
    }
}
```

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

Ist der Code valide?

```
abstract class Werkzeug { }
public class Main {
   public static void main(String[] args) {
      Werkzeug w = new Werkzeug(); // FEHLER!
   }
}
```

- sind Klassen, die nicht instanziiert werden können
- dienen als Basis
- können Datenfelder haben
- können einen Konstruktor oder mehrere Konstruktoren haben
- können abstrakte und konkrete Methoden haben
  - Sobald eine Methode innerhalb einer Klasse abstrakt ist, muss die Klasse selbst als abstrakt markiert werden!

Abstrakte Klassen können nicht instanziiert werden!

Können abstrakte Klassen final sein?

- Nein! – final und abstrakt sind nämlich Gegensätze
- final beschreibt, dass eine Klasse in ihrer Implementierung nicht mehr geändert werden kann
- abstrakt beschreibt, dass eine Klasse erweitert, also geändert werden kann

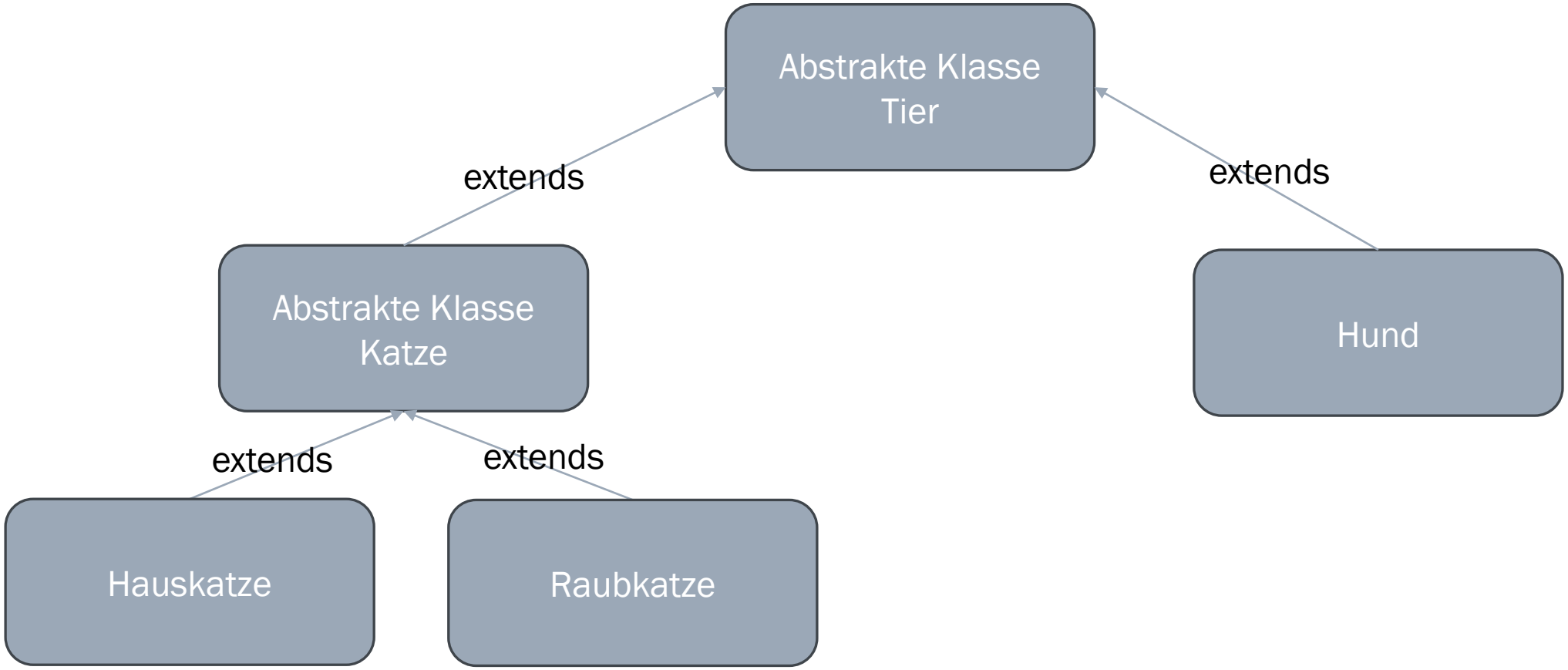
Warum abstrakte Klassen?

- ermöglichen Code-Wiederverwendung
- definieren eine allgemeine Struktur
- verhindern die Instanziierung von Klassen, die nur als Basis dienen sollen

- Wie nutzen ohne Instanziierung/Objekterstellung? Wie kann man Code wiederverwenden?
  - Mittels Vererbung

SMART INDUSTRY CAMPUS

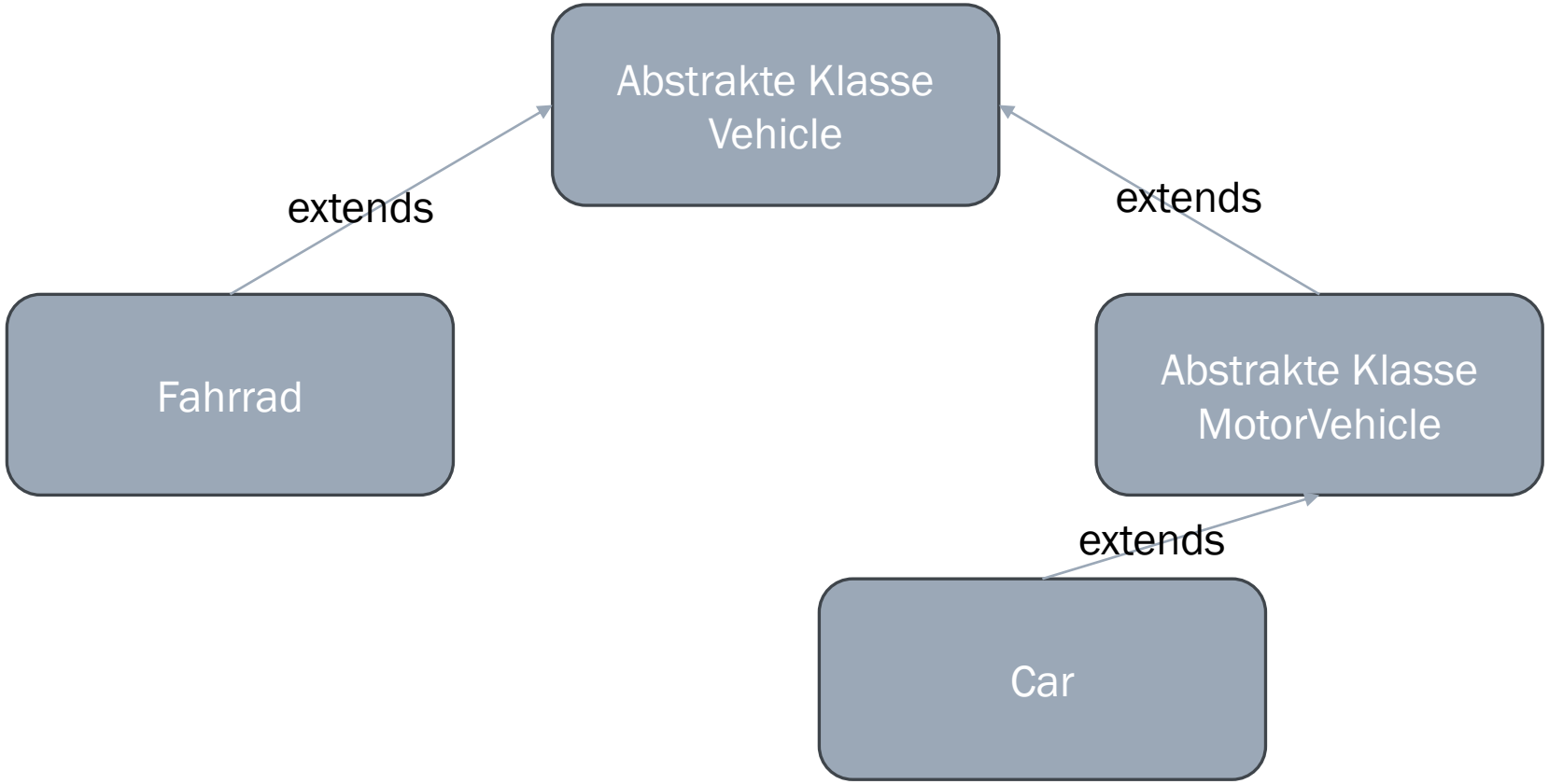
Abstrakte Klassen und Vererbung



Abstrakte Klassen und Vererbung - Zusammenfassung

SMART INDUSTRY CAMPUS

- Ist eine Unterklasse abstrakt, müssen die Methoden der Basisklasse nicht direkt implementiert werden
- Die erste konkrete Unterklasse muss alle noch nicht implementierten Methoden implementieren
- die Unterklasse kann eigene Methoden hinzufügen

Weiteres Beispiel zum experimentieren, 30 Minuten

SMART INDUSTRY CAMPUS